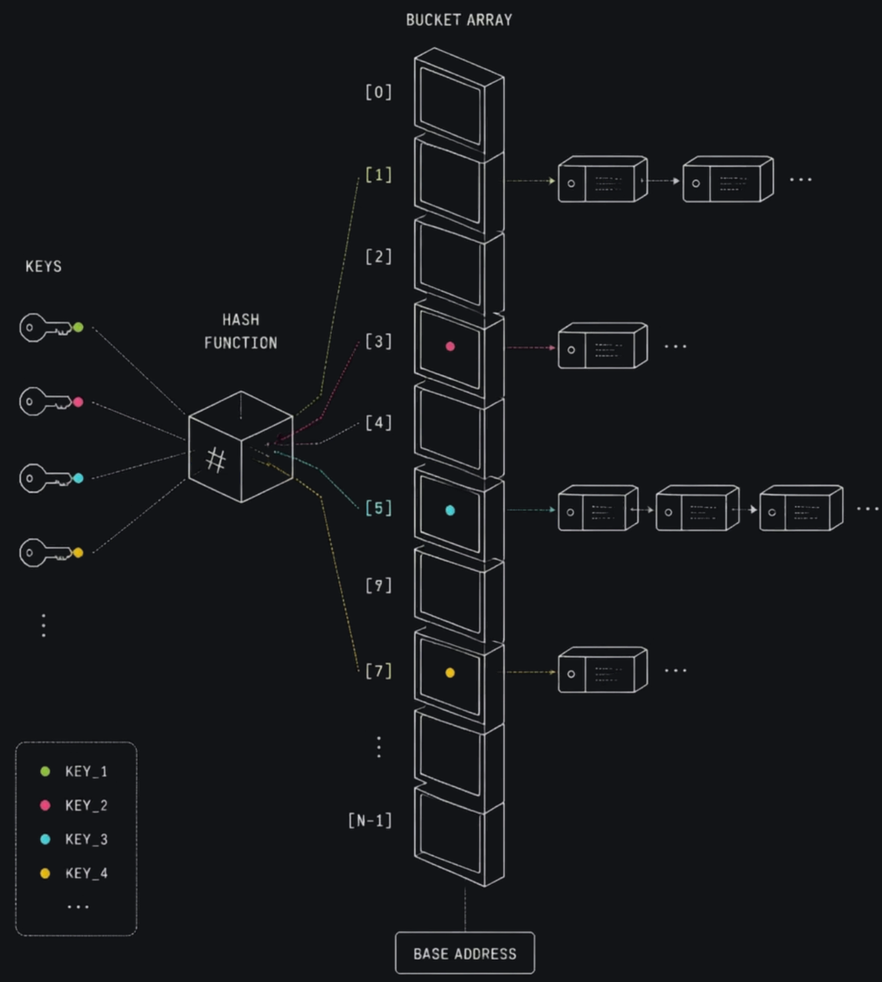
CONFIGURAÇÃO 72
TABELA HASH: DA CONFIGURAÇÃO À EVIDÊNCIA

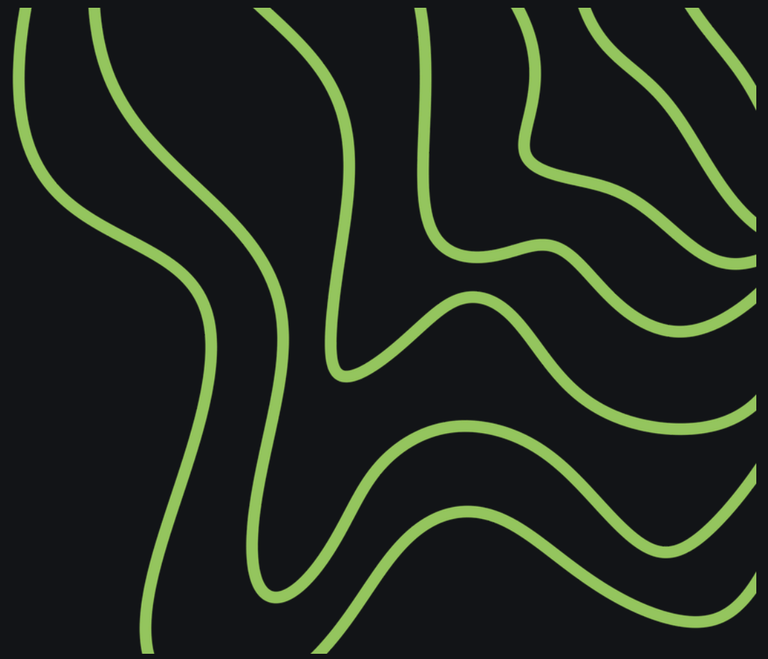
Universidade Federal Rural do Semi-Árido
Centro Multidisciplinar de Pau dos Ferros
Departamento de Engenharias e Tecnologia
Componente:Laboratório de Algoritmo e Estruturas de Dados 2
Docente:Kennedy Reurison Lopes
Discentes:Francisco Eduardo,Kaualy Costa, Pedro Gabriel Alves



ROTEIRO DA APRESENTAÇÃO

TÓPICOS DA APRESENTAÇÃO

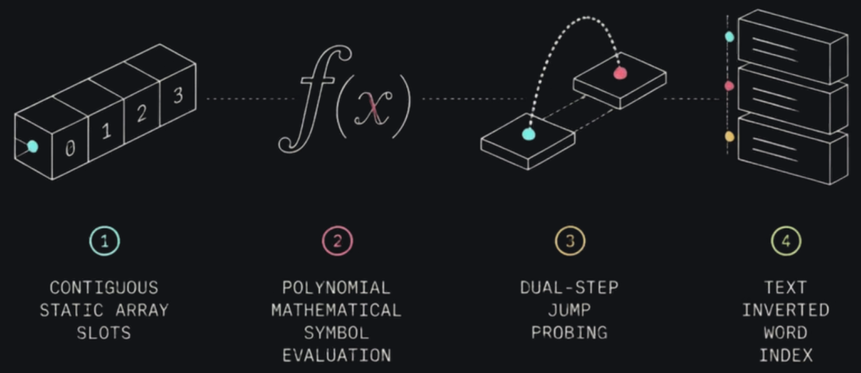
01	Configuração e Normalização
02	Estrutura em C e Hash Polinomial
03	Colisões por Hash Duplo
04	Testes e Análise de Desempenho



A CONFIGURAÇÃO ATRIBUÍDA DEFINE O PROJETO

CONFIGURAÇÃO 72

E1.1	Vetor estático
H6	Hash polinomial
C4	Hash duplo
A4	Índice de palavras



O DESAFIO É CONSTRUIR A ESTRUTURA POR CONTA PRÓPRIA

Proibido o uso de coleções nativas como dicionários ou tabelas prontas.



SEM BIBLIOTECAS

Zero uso de dict, HashMap ou unordered_map na sustentação da tabela principal.



CRUD COMPLETO

Rotinas manuais de inserção, busca, remoção lógica (tombstone), atualização e iteração.



TRANSPARÊNCIA TOTAL

Método exibirEstrutura() implementado para auditar ocupação, colisões e saltos na depuração.

COMO A CHAVE VIRA UMA POSIÇÃO

01 Normalizar a palavra: caixa, pontuação e acentos

02 Converter a chave textual de forma determinística

03 Aplicar a função hash e obter um índice entre 0 e m - 1

Chave → hash → índice → registro

H6 COMBINA OS SÍMBOLOS EM UMA FUNÇÃO POLINOMIAL

A função polinomial pondera cada caractere por potências crescentes de uma base constante p, dispersando anagramas.

FORMULAÇÃO MATEMÁTICA

$$h(k) = \left(\sum_{i=0}^{L-1} c_i \cdot p^{L-1-i} \right) \bmod m$$

Onde c_i é o código ASCII do i-ésimo caractere normalizado e $p = 31$ é o número primo base.

EXEMPLO DE CÁLCULO: "HASH"

'h'=104, 'a'=97, 's'=115, 'h'=104

H = 104·31³ + 97·31² + 115·31¹ + 104

H = 3.098.264 + 93.217 + 3.565 + 104

Valor Base = 3.195.150

Se $m = 1013$ → **Posição = 148**

PROPRIEDADE · Avaliação feita via regra de Horner com aritmética modular para mitigar overflow inteiro.

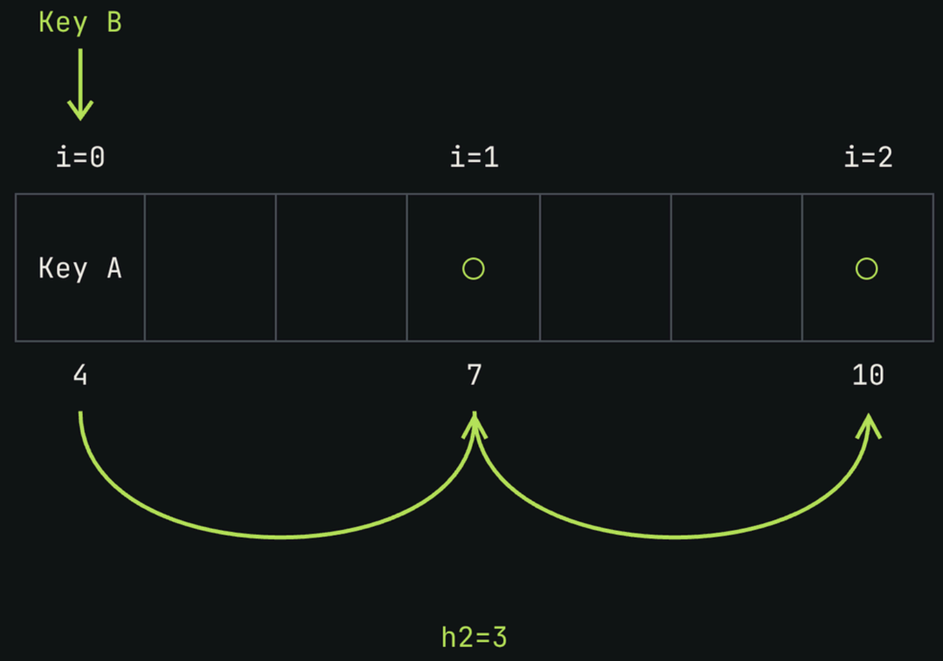
C4 RESOLVE COLISÕES COM HASH DUPLO

Em caso de ocupação no índice primário, o salto de sondagem é orientado por uma segunda função independente $h_2(k)$.

$$p_i(k) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Função de salto: $h_2(k) = 1 + (k \bmod (m-1))$

O termo $+1$ garante salto não nulo ($h_2(k) > 1$), enquanto $\gcd(h_2(k), m) = 1$ assegura que toda a tabela seja percorrida sem loops curtos.



A4 TRANSFORMA A TABELA EM UM ÍNDICE DE PALAVRAS

Cada registro armazena a contagem de termos e as referências de localização.

CHAVE

Palavra normalizada

Termo textual após conversão minúscula, remoção de pontuação e diacríticos.

Exemplo: "Computação" → "computacao"

VALOR

Frequência e ocorrências

Contador de frequência acumulada e ocorrências do termo.

Exemplo: freq: 14, linhas: [3, 19, 42]

TRATAMENTO DE TEXTO

Regras léxicas

Definir caixa, pontuação, acentos e separação das palavras. Termos idênticos incrementam metadados sem duplicar slots.

Chave única preservada por slot.

ESTRUTURA DE DADO · Registro: `struct RegistroIndice { string termo; int frequencia; ListaEncadeada linhas; };`

OS TESTES PRECISAM PROVAR CORREÇÃO, NÃO SÓ EXECUÇÃO

Buscar existente e inexistente

Recusar duplicatas e atualizar sem trocar a chave

Forçar colisões e remover uma delas

Confirmar que as demais continuam acessíveis

Verificar tabela cheia e condições-limite

A ANÁLISE CONECTA CARGA, COLISÕES E TEMPO

Resultados coletados com 5 repetições por cenário para avaliar o impacto do fator de carga α = n/m:

REGISTROS	CAPACIDADE	α	COLISÕES	INSERÇÃO (MS)	BUSCA OK (MS)	BUSCA AUSENTE (MS)
100	211	0,474	25,0	0,008	0,009	0,010
1.000	2003	0,499	267,4	0,099	0,104	0,116
10.000	20011	0,500	2515,2	0.785	0,853	0,981

O RESULTADO DEVE DEFENDER AS DECISÕES TÉCNICAS

EVIDÊNCIA & CONFORMIDADE

INSPEÇÃO VISUAL

exibirEstrutura()

Apresenta o vetor, identificando slots vazios, ocupados e lápides de remoção.

FIDELIDADE AO SORTEIO

Configuração 72 Cumprida

Integração coesa entre E1.1, H6, C4 e A4 sem desvios de especificação.

INSIGHTS & EVOLUÇÃO

IMPACTO DE α

Curva de Desempenho

Colisões sobem rapidamente quando $\alpha > 0,7$, encarecendo a sondagem no hash duplo.

LIMITAÇÕES & FUTURO

Próximos Passos

Implementação de redimensionamento dinâmico *rehashing* ( ) para sustentar tempos constantes.

>_ A implementação do zero valida na prática o compromisso entre dispersão, custo de memória e garantia de acesso determinístico.

Obrigado!